

GestorCan

Documento tecnico - Backend FastAPI

Version 1 - Inicio del backend cloud conectado a Railway y PostgreSQL

Objetivo: dejar documentada la base tecnica para construir el producto paso a paso, con estructura clara y preparada para crecer.

1. Resumen del backend

El backend de GestorCan es el cerebro de la aplicacion. En esta primera fase se ha creado una API con FastAPI, desplegada en Railway y conectada a una base de datos PostgreSQL tambien alojada en Railway.

La API ya responde desde internet y sirve como base para construir los modulos reales: clientes, mascotas, entradas, calendario, facturacion e informes.

Elemento	Decision actual	Motivo
Lenguaje	Python	Continuidad con la experiencia previa y rapidez de desarrollo.
Framework backend	FastAPI	API moderna, documentacion automatica y buen rendimiento.
Base de datos	PostgreSQL	Robusta, profesional y preparada para SaaS.
Alojamiento	Railway	Despliegue rapido desde GitHub para la fase inicial.
Repositorio	gestorcan-backend	Codigo separado y preparado para despliegue continuo.

2. Estructura inicial del repositorio

La estructura inicial es deliberadamente sencilla. En esta fase interesa que funcione bien antes de dividirlo en muchos modulos.

```
gestorcan-backend/  
|-- main.py  
|-- database.py  
|-- models.py  
|-- requirements.txt
```

Archivo	Funcion
main.py	Punto de entrada de FastAPI. Crea la aplicacion, registra rutas y arranca las tablas.
database.py	Configura la conexion con PostgreSQL usando la variable DATABASE_URL.
models.py	Define los modelos de datos iniciales con SQLAlchemy.
requirements.txt	Lista las dependencias necesarias para que Railway instale el proyecto.

3. Dependencias del proyecto

El archivo requirements.txt permite que Railway instale automaticamente las librerias necesarias durante el despliegue.

```
fastapi
uvicorn
sqlalchemy
psycopg2-binary
python-dotenv
```

Explicacion rapida

Dependencia	Para que sirve
fastapi	Framework principal para crear la API.
uvicorn	Servidor ASGI que ejecuta la aplicacion FastAPI.
sqlalchemy	ORM para trabajar con la base de datos desde Python.
psycopg2-binary	Driver para conectar Python con PostgreSQL.
python-dotenv	Ayuda a gestionar variables de entorno en desarrollo local.

4. Conexion a PostgreSQL

La conexion con la base de datos se realiza mediante una variable de entorno llamada DATABASE_URL. Esta variable no debe escribirse fija en el codigo, porque contiene credenciales.

```
import os
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = os.getenv("DATABASE_URL")

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)

Base = declarative_base()
```

Punto clave

Si DATABASE_URL no existe en Railway, el backend arranca y se cae inmediatamente. Este fue el motivo del primer estado Crashed. La solucion fue crear la variable DATABASE_URL en el servicio gestorcan-backend usando el valor generado por PostgreSQL.

5. Primer modelo de datos

El primer modelo creado fue Cliente. Es una version minima para comprobar que SQLAlchemy puede crear tablas en PostgreSQL.

```
from sqlalchemy import Column, Integer, String
from database import Base

class Cliente(Base):
    __tablename__ = "clientes"

    id = Column(Integer, primary_key=True, index=True)
    nombre = Column(String)
    telefono = Column(String)
```

Nota de desarrollo

Este modelo es solo una base. Mas adelante se ampliara con campos reales como DNI/NIF, email, direccion, observaciones, estado y fecha de alta.

6. Aplicacion FastAPI inicial

El archivo main.py crea la aplicacion, genera las tablas y define endpoints de prueba.

```
from fastapi import FastAPI
from database import engine, Base
from models import Cliente

app = FastAPI()

Base.metadata.create_all(bind=engine)

@app.get("/")
def root():
    return {"mensaje": "GestorCan funcionando"}

@app.get("/clientes")
def obtener_clientes():
    return {"mensaje": "Listado de clientes"}
```

El endpoint raiz se ha usado para confirmar que el backend esta funcionando correctamente desde internet.

7. Despliegue en Railway

Railway se conecta con el repositorio gestorcan-backend en GitHub. Cada despliegue toma el código del repositorio, instala las dependencias de requirements.txt y ejecuta el comando de arranque.

Comando de arranque

```
uvicorn main:app --host 0.0.0.0 --port $PORT
```

El uso de \$PORT es importante porque Railway asigna el puerto dinámicamente. La aplicación no debe depender de un puerto fijo en producción.

Servicios actuales en Railway

Servicio	Estado	Funcion
gestorcan-backend	Online	Ejecuta FastAPI y expone la API pública.
Postgres	Online	Base de datos PostgreSQL para persistencia.

8. Dominio público de Railway

Se generó un dominio público para el backend desde Settings - Networking. El backend responde correctamente con el mensaje de prueba.

Respuesta esperada en /

```
{"mensaje": "GestorCan funcionando"}
```

También queda disponible la documentación interactiva de FastAPI en la ruta /docs.

9. Estado actual del backend

Area	Estado
Repositorio GitHub	Creado y conectado a Railway.
Backend FastAPI	Online.
PostgreSQL	Online.
DATABASE_URL	Configurada en el servicio backend.
Endpoint raiz /	Funcionando.
Endpoint /docs	Disponible para probar la API.

10. Proximos pasos tecnicos

El siguiente objetivo es convertir el endpoint de clientes en una funcionalidad real de lectura y escritura contra PostgreSQL.

Paso	Objetivo
1	Crear schemas con Pydantic para validar datos de clientes.
2	Crear dependencia get_db para abrir y cerrar sesiones de base de datos.
3	Crear POST /clientes para guardar clientes reales.
4	Crear GET /clientes para listar clientes desde PostgreSQL.
5	Probar ambos endpoints desde /docs.
6	Documentar el primer CRUD real de GestorCan.

11. Criterio de trabajo

La filosofia de desarrollo para GestorCan sera avanzar por hitos pequenos y documentados. Cada cambio importante debe quedar registrado: que se hizo, por que se hizo y como se comprueba que funciona.

Esto evita perder informacion, facilita volver atras cuando aparezca un error y convierte el proyecto en un producto mantenible. En cristiano: menos caos y menos exorcismos de bugs a las dos de la mañana.

12. Checklist de verificacion

Comprobacion	Resultado esperado
Backend en Railway	Estado Online.
Postgres en Railway	Estado Online.
Variable DATABASE_URL en backend	Existe y apunta a PostgreSQL.
Ruta /	Devuelve mensaje GestorCan funcionando.
Ruta /docs	Muestra Swagger UI de FastAPI.

Documento generado como parte de la documentacion inicial de GestorCan.